Task 8

Document the Complete AI Pipeline Architecture and Workflow

Predictive Analytics & AI Model Deployment - Customer Churn Prediction

Objective. Document the architecture, workflow, design decisions, reproduction steps and limitations of the full AI pipeline.

1. Architecture

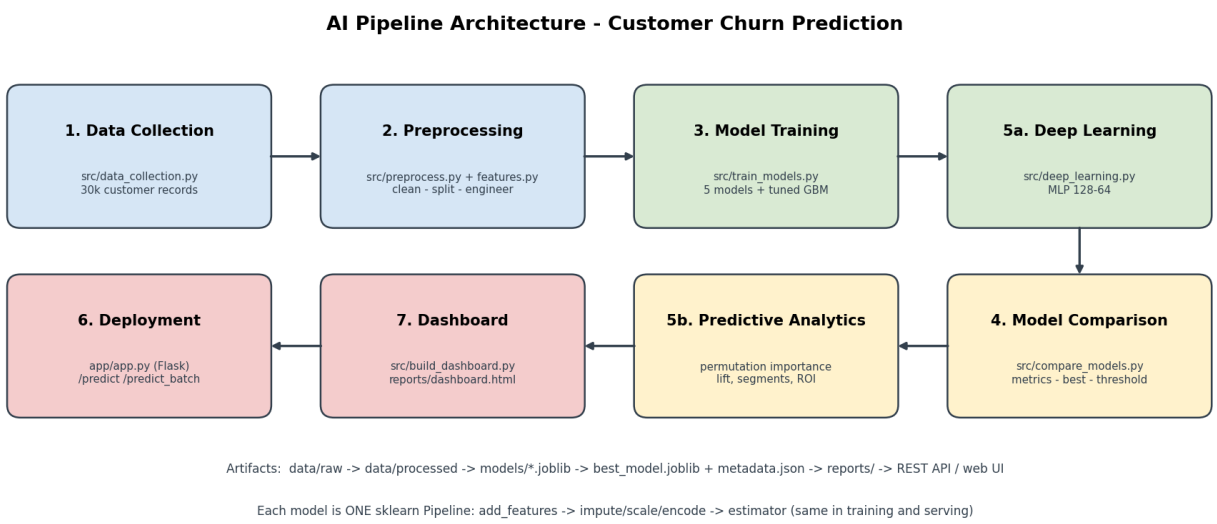


Figure 1 - AI pipeline architecture

2. Workflow

#	Stage	Input	Output	Module
1	Data collection	generator / real CSV	data/raw csv	data_collection.py
2	Preprocessing	raw csv	train.csv, test.csv, EDA figures	preprocess.py, features.py
3	Model training	train.csv	6 fitted pipelines	train_models.py
5a	Deep learning	train.csv	MLP pipeline	deep_learning.py
4	Model comparison	test.csv + models	best_model.joblib, metadata.json	compare_models.py
5b	Predictive analytics	best model + test	importance, lift, segments, ROI	predictive_analytics.py
6	Deployment	best_model.joblib	REST API + web UI	app/app.py
7	Dashboard	results + figures	dashboard.html	build_dashboard.py

3. Design principles

- **One pipeline object per model** (features + preprocessing + estimator). Training, cross-validation and serving use identical code, eliminating train/serve skew and leakage.
- **Held-out test set** used only for final comparison; model threshold selected from out-of-fold training predictions.

- **Reproducibility**: fixed seeds, one command (`python run_pipeline.py`) rebuilds all data, models, figures and the dashboard.
- **Separation of concerns**: config, features, training, evaluation, serving and reporting are independent modules.
- **Defensive serving**: input validation, tolerant preprocessing (unknown categories ignored, invalid numbers imputed), JSON errors.

4. Project structure

```
churn_ml_project/
|-- run_pipeline.py            run Tasks 1-5, 7, 8 end-to-end
|-- requirements.txt  Dockerfile  Procfile  README.md
|-- src/
|   |-- config.py              paths, constants, risk bands
|   |-- features.py            cleaning + feature engineering + sklearn pipeline
|   |-- data_collection.py     Task 1        preprocess.py        Task 2
|   |-- train_models.py        Task 3        compare_models.py    Task 4
|   |-- deep_learning.py       Task 5a       predictive_analytics.py  Task 5b
|   |-- predict.py             inference     build_dashboard.py   Task 7
|   `-- build_docs.py / build_pdfs.py        documentation builders
|-- app/  app.py, templates/index.html       Task 6 (Flask API + UI)
|-- tests/test_api.py
|-- data/ raw/  processed/     models/ *.joblib, best_model.joblib, metadata.json
|-- results/  JSON/CSV outputs   reports/ figures/, dashboard.html
`-- docs/ architecture.png, ARCHITECTURE.md, pdfs/
```

5. How to reproduce

```
pip install -r requirements.txt
python run_pipeline.py            # ~2 minutes
python -m unittest tests.test_api -v
python -m app.app                 # http://127.0.0.1:5000
```

6. Final results at a glance

Item	Value
Dataset	30,000 customers, 18 columns (synthetic), churn 26.9%
Features	16 raw + 9 engineered -> 45 model inputs after encoding
Models trained	7 (5 classical, 1 tuned boosting, 1 neural network)
Selected model	Logistic Regression, threshold 0.30
Test ROC-AUC / PR-AUC	0.791 / 0.577
Test recall / precision / F1	0.68 / 0.49 / 0.57
Top-decile lift	2.58x

7. Limitations and next steps

- **Synthetic data**: results demonstrate a working pipeline, not real-world accuracy. Retrain on real data before any business decision.
- Deep learning uses scikit-learn's MLP because TensorFlow/PyTorch were unavailable; a Keras model can replace the estimator (e.g. via SciKeras).
- ROI figures are placeholder assumptions. The API has no authentication, rate limiting or monitoring yet.
- Next: real data, drift monitoring, MLflow model registry, CI/CD, authentication, A/B test of retention offers.

8. GitHub repository

A local git repository with an initial commit is included in the project folder. To publish it:

```
cd churn_ml_project
git remote add origin https://github.com/<your-username>/<repo-name>.git
git branch -M main
git push -u origin main
```

The public GitHub URL cannot be created from this environment (no internet / no account access); create an empty repository on GitHub and run the commands above.

Appendix - Source code

run_pipeline.py

```
"""Run the complete pipeline end-to-end:  python run_pipeline.py"""
import importlib
import time

STEPS = [("Task 1  Data collection", "src.data_collection"),
         ("Task 2  Preprocessing & feature engineering", "src.preprocess"),
         ("Task 3  Train multiple models", "src.train_models"),
         ("Task 5a Deep learning (MLP)", "src.deep_learning"),
         ("Task 4  Compare models / select best", "src.compare_models"),
         ("Task 5b Predictive analytics", "src.predictive_analytics"),
         ("Task 7  Dashboard", "src.build_dashboard"),
         ("Task 8  Architecture diagram", "src.build_docs")]

if __name__ == "__main__":
    for title, mod in STEPS:
        t0 = time.time(); print(f"\n=== {title} ===")
        importlib.import_module(mod).main()
        print(f"[done in {time.time() - t0:.0f}s]")
    print("\nAll done. Start the API with:  python -m app.app   (or gunicorn)")
```

src/config.py

```
"""Central configuration: paths, constants and shared settings."""
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
DATA_RAW = ROOT / "data" / "raw"
DATA_PROC = ROOT / "data" / "processed"
MODELS = ROOT / "models"
RESULTS = ROOT / "results"
REPORTS = ROOT / "reports"
FIGS = REPORTS / "figures"
DOCS = ROOT / "docs"

for _p in (DATA_RAW, DATA_PROC, MODELS, RESULTS, FIGS, DOCS):
    _p.mkdir(parents=True, exist_ok=True)

TARGET = "churn"
ID_COL = "customer_id"
SEED = 42
RAW_FILE = DATA_RAW / "customer_churn_raw.csv"

# Risk segmentation used by the dashboard and the API
RISK_BINS = [0.0, 0.25, 0.55, 1.0001]
RISK_LABELS = ["Low", "Medium", "High"]
```